

UNIT 1, FOUNDATIONS OF AI AGENTS AND MICROSOFT FOUNDRY

LECTURE 5

Agent Lifecycle and Design Principles

You can build one now. This is about everything around it: what to decide before you write code, and the principles that separate a system somebody maintains from one they rewrite.

SECTION 00

Where you are

Four lectures in, and you can already build the thing.

RECALL

What you can already do

LECTURE 1

Build one

Goal, tools, loop, whitelist, budget. Forty lines, no framework.

LECTURE 2

Choose its shape

Reflex, model based, goal based, utility based. The environment picks the rung.

LECTURE 3

Keep it affordable

Measure the transcript, trim it, pin what must survive.

SO WHAT IS LEFT

Knowing how to build one is not the same as knowing what to build, or in what order, or when to stop. Today is about the decisions that surround the code, and they are the ones that decide whether anyone can maintain what you wrote.

Today

- The lifecycle. Nine stages from an idea to something running, and where students actually start.
- The decisions worth making before you write any code, as a checklist you can run.
- Ten design principles, all of which you have already met as a failure in a previous lecture.
- Four anti patterns, each of which I have seen in a submitted practical.
- A linter for your own agent. It reads your tool schema and tells you what is wrong with it.

HOLD ON TO THIS

Nothing today calls a model. The whole lecture runs offline and costs nothing, which is itself a small lesson about how much of this work is design rather than inference.

SECTION 01

The lifecycle

Nine stages, and most people start at stage five.

Nine stages

1 Define the job

PEAS. What counts as doing well.

2 Choose the shape

Workflow, router or agent.

3 Design the tools

One job each. Descriptions are instructions.

4 Write instructions

System prompt. Refusals included.

5 Build the loop

With every guard from Lectures 1 to 4.

6 Evaluate

Against stage 1. Unit 5.

7 Deploy

Somewhere other than your laptop. Unit 5.

8 Observe

Traces, cost, failure rates. Unit 5.

9 Iterate or retire

Most agents should be retired sooner.

Stages 1 to 5 are this unit. Stages 6 to 9 are Unit 5, and they are half the job.

Where people actually start

Stage 5. Open the editor, import the SDK, start writing the loop.

It feels productive because code appears. Then stage 1 gets decided later, accidentally, halfway through debugging something else, and the answer is whatever makes the current bug go away.

THE COST OF SKIPPING STAGE 1

If you never wrote down what counts as doing well, you cannot tell whether your agent works. You will end up judging it on whether the demo went smoothly, which is the least informative test available and the one every student defaults to.

The decisions worth making first

```
spec = AgentSpec(  
    name="hotel-assistant",  
    performance_measure="Finds an available room within budget",  
    environment="Three hotels, rates, distances, availability",  
    actuators="list_hotels, get_room_availability, get_details",  
    sensors="Tool results plus the conversation so far",  
    shape="agent", strategy="react", rung="utility_based",  
    what_it_refuses=["payment", "medical or legal advice"],  
    pinned_facts=["stated budget", "stated dates"],  
    failure_behaviour="Says it cannot match and names what it checked",  
    max_steps=6,  
)
```

**EVERY BLANK FIELD IS A DECISION YOU WILL MAKE
ANYWAY**

Later, accidentally, under pressure, in the middle of fixing something else. Making them deliberately, in one place, before any code exists, costs twenty minutes.

SECTION 02

Ten design principles

Every one of these you have already met as a failure.

Principles one to three, the tools

1

One tool, one job

A tool that checks availability and computes price and sends confirmation gets called at the wrong moment. Lecture 1.

2

Return readable strings

The result goes back to the model as text anyway. A readable string saves a serialisation step and makes traces legible at 2am.

3

The description is an instruction

It is the only thing the model routes on. Vague description, wrong tool, and the bug is in your English rather than your Python. Lecture 1.

ALL THREE ARE CHECKABLE WITHOUT RUNNING ANYTHING

Which is why the linter in section four can find all of them by reading your schema. If a rule can be checked statically, it should be, because a rule nobody checks is a rule nobody follows.

Principles four to six, the loop

4

Enforce in code, request in prompt

A prompt is a request the model usually honours. Code is a guarantee. When correctness matters the difference is not subtle. Lecture 1 and 3.

5

Fail honestly, not fluently

An agent with nothing to say will still produce fluent text. Decide in advance what it says when it cannot answer. Lecture 3.

6

Bound everything

Steps, tokens, spend, wall clock. Every one of those has an unbounded failure mode and you have watched two of them happen live.

PRINCIPLE FIVE IS THE ONE PEOPLE SKIP

Because it only shows up when things go wrong, and the demo never goes wrong. An agent that says "I checked all three hotels and none has a room in your range" is more useful than one that invents a fourth hotel, and only one of those is the default.

Principles seven to ten, the system

7

Least privilege from day one

The environment is exactly the tool list. Add a tool and you have widened the blast radius. Lecture 2, and all of Unit 6.

8

Make it traceable

If you cannot show one request moving through the system, you cannot debug it and you cannot defend it in a viva.

9

Make retries safe

Agents retry. If a tool books a room, being called twice books two rooms. Read only tools are safe, writing tools need care.

HOLD ON TO THIS

Principle nine is the one that has not bitten you yet, because every tool in this course so far is read only. It will bite in Unit 4.

SECTION 03

Four anti patterns

All four have appeared in submitted work.

The god tool, and the prompt that grew

ANTI PATTERN 1

The god tool

handle_booking(action, params). One tool, a string switch inside, six behaviours. The model has to guess which mode applies and it will guess wrong under load.

Symptom: the tool name contains "and", "handle", "process" or "manage".

ANTI PATTERN 2

The prompt that grew

Every edge case became another sentence. Nine hundred words, nobody knows which lines still matter, and all of it is billed on every single turn.

Symptom: nobody on the team will delete a line because nobody knows what it was for.

BOTH HAVE THE SAME CAUSE

A new requirement arrived and the cheapest place to put it was somewhere that already existed. That is a reasonable instinct and it compounds badly, which is why stage 1 of the lifecycle is worth twenty minutes.

The demo shaped agent, and the unbounded budget

ANTI PATTERN 3

The demo shaped agent

Works beautifully on the one question it was built against. Every other question produces something confident and wrong.

Symptom: the only test anyone has run is the one from the video.

ANTI PATTERN 4

The unbounded budget

No step cap, no token cap, no spend cap, no detector. It works until a question it cannot answer arrives, and then it works very expensively.

Symptom: nobody can say what the worst case cost of one request is.

A QUESTION YOU SHOULD BE ABLE TO ANSWER ABOUT YOUR OWN AGENT

What is the most this can cost me for a single request, and what does it say when it fails? If you cannot answer both in one sentence each, you have anti pattern three or four, and probably both.

SECTION 04

A linter for your agent

Design rules nobody checks are design rules nobody follows.

Static review, no model calls

```
from cse476.design import audit_tools, audit_guards, report

print(report(audit_tools(TOOL_SCHEMA, REGISTRY), "my agent"))
print(report(audit_guards(max_steps=6, has_whitelist=True), "guards"))
```

ERROR In the schema but not the registry, or the reverse

ERROR Schema declares a parameter the function does not accept

ERROR No description, no parameter type, no step budget, no whitelist

WARNING Description under forty characters, or a name suggesting two jobs

NOTE No docstring, returns something other than a string, nothing pinned

The linter had a bug, and that is the lesson

The first version reported this, on correct code:

```
[note  ] get_nightly_rate      Returns str rather than str.
```

The cause: ``from __future__ import annotations`` turns every annotation into a string, so comparing against the type ``str`` fails even when the function is annotated correctly.

WHY THIS IS ON A SLIDE

A linter that nags about correct work gets switched off, and then it catches nothing at all. This is the same principle as the no progress detector in Lecture 4: you must test that a check stays quiet, not only that it fires. The test suite now has a case for exactly this false positive.

HOLD ON TO THIS

Also worth saying plainly: the fixed linter then found three real problems in my own teaching code. I fixed the code rather than the linter.

Hold on to this

Every decision you skip gets made anyway, later, by accident, under pressure.

Start at stage one, not stage five

Twenty minutes of PEAS saves days of debugging.

A rule nobody checks is not a rule

If it can be checked statically, check it in code.

Test that a check stays quiet

A linter that nags about correct work gets switched off.

Know your worst case cost

And know what it says when it cannot answer.

Before next session

DO

Audit your own agent

Run `audit_tools` and `audit_guards` against your Practical 2 code. Fix every error. Justify or fix every warning.

DO

Write your AgentSpec

Fill it in until `is_ready()` returns `True`. This becomes the design section of your capstone proposal.

THINK

Find an anti pattern

In your own code, not somebody else's. Name which of the four, and what you would change.

ON THE FIRST ONE

Warnings are not automatically wrong. A short description may be fine for an obvious tool. But you have to say why, in a comment, rather than ignoring it. An unexplained warning and a fixed one are worth the same marks. An ignored one is not.

Next session

Unit 1, Lecture 6

Microsoft Foundry, First Contact

Five lectures of building agents by hand, on purpose. Now we open the enterprise platform and you will recognise every part of it, because you have already written each one yourself. Projects, the model catalogue, deployments, and why it has been renamed twice in twelve months.

COME WITH

Your audit output, your completed AgentSpec, and one anti pattern you found in your own code.